

Ôn tập Phỏng vấn Tester Ngân hàng

Dựa trên dự án thật: Fintech Core Service Layer (hệ thống lỗi kiểu T24)

Phiên bản	1.0
Ngày	2026-09-15
Cách dùng	Không học thuộc lòng — hiểu bản chất rồi kể lại bằng chính ví dụ trong dự án này . Nhà tuyển dụng phân biệt rất rõ người học vẹt lý thuyết và người đã thực sự "sờ" vào một hệ thống ngân hàng thật, dù chỉ là bản demo.

Tài liệu tham chiếu cùng bộ: `SRS.md` (đặc tả hệ thống), `TEST_DESIGN.md` (146 test case + SQL), `ANSWER_KEY.md` (đáp án mẫu). Khi phỏng vấn hỏi sâu, có thể mở các file này ra minh hoạ trực tiếp.

PHẦN A — Lý thuyết kiểm thử phần mềm (nền tảng bắt buộc)

A.1 SDLC vs STLC

	SDLC (SOFTWARE DEVELOPMENT LIFE CYCLE)	STLC (SOFTWARE TESTING LIFE CYCLE)
Phạm vi	Toàn bộ vòng đời phát triển	Chỉ phần kiểm thử, chạy song song với SDLC
Các bước	Requirement → Design → Development → Testing → Deployment → Maintenance	Requirement Analysis → Test Planning → Test Case Design → Test Environment Setup → Test Execution → Test Closure

Ví dụ kể trong phỏng vấn: "Trong dự án core banking em làm, giai đoạn Test Case Design em bám sát `SRS.md` — với mỗi module như Tiền gửi có kỳ hạn, em không chỉ viết test case chức năng mà còn phải phân tích kỹ business rule (lãi đơn/lãi kép, phạt lãi khi rút trước hạn) trước khi viết test, vì nếu hiểu sai công thức thì test case sai mà vẫn tưởng đúng."

A.2 Test Levels (Mức độ kiểm thử)

LEVEL	MÔ TẢ	VÍ DỤ TRONG DỰ ÁN
Unit Testing	Test 1 hàm/module độc lập (thường do Dev làm)	Hàm <code>simpleInterest()</code> , <code>compoundInterestFullTerm()</code> trong <code>termDepositMath.ts</code>
Integration Testing	Test sự phối hợp giữa các module	Mở sổ tiết kiệm → phải trừ đúng balance account và tạo đúng bút toán GL — 2 module (Term Deposit + General Ledger) phải khớp nhau

LEVEL	MÔ TẢ	VÍ DỤ TRONG DỰ ÁN
System Testing	Test toàn bộ hệ thống end-to-end	Kịch bản: khách mở tài khoản → vay tiền → trả góp → tắt toán, tất cả qua API thật
UAT (User Acceptance Testing)	Khách hàng/end-user xác nhận hệ thống đáp ứng nhu cầu thật	Với hệ thống này: nghiệp vụ "maker nộp lệnh, checker duyệt" phải đúng quy trình vận hành ngân hàng thật, không chỉ đúng code

A.3 Test Types (Loại kiểm thử)

- **Functional Testing** — đúng nghiệp vụ (vd: rút trước hạn có tính đúng lãi phạt không).
- **Non-functional Testing:**
 - *Performance* — hệ thống chịu tải bao nhiêu giao dịch/giây (dự án demo **chưa test phần này** — biết nói thật trong phỏng vấn là điểm cộng, đừng bịa).
 - *Security* — RBAC, JWT, SQL injection, có bị bypass Maker-Checker không.
 - *Usability* — giao diện Console có dễ dùng cho teller không.
- **Regression Testing** — sau khi sửa 1 bug (vd thêm Idempotency-Key cho loan), phải chạy lại các test case liên quan để chắc chắn không phá vỡ luồng cũ.
- **Smoke Testing** — kiểm tra nhanh "hệ thống có chạy được không" trước khi test sâu (vd: `GET /` trả 200, `GET /docs` load được).
- **Sanity Testing** — kiểm tra nhanh 1 tính năng cụ thể vừa sửa còn hoạt động đúng cơ bản không.
- **Regression vs Retesting:** Retesting = test lại **đúng** case đã fail để xác nhận bug đã sửa; Regression = test **thêm** các case khác xung quanh để chắc chắn bản sửa không gây tác dụng phụ.

A.4 Kỹ thuật thiết kế test case (Test Design Techniques)

KỸ THUẬT	KHI DÙNG	VÍ DỤ THẬT TRONG DỰ ÁN
Equivalence Partitioning	Chia input thành vùng tương đương, chỉ cần test đại diện	<code>currency</code> khớp/lệch tài khoản; <code>amount</code> dương/âm/0
Boundary Value Analysis (BVA)	Test đúng ranh giới, vì bug hay nằm ở biên	Ngưỡng gian lận: <code>amount = threshold</code> (không tạo alert) vs <code>threshold+1</code> (tạo alert); nợ xấu: quá hạn đúng 90 ngày (không defaulted) vs 91 ngày (defaulted)
Decision Table	Nhiều điều kiện kết hợp cho ra nhiều kết quả	Ma trận Maker-Checker: (role × có phải người nộp lệnh) → 4 tổ hợp, mỗi tổ hợp 1 kết quả khác nhau
State Transition Testing	Hệ thống có máy trạng thái	<code>accounts.status</code> , <code>loans.status</code> , <code>term_deposits.status</code> , <code>auth_queue.status</code> — mỗi bảng trong <code>TEST_DESIGN.md</code> mục 6
Error Guessing	Dựa kinh nghiệm đoán chỗ dễ sai	Đoán được lỗi "currency giả mạo" (khai USD cho tài khoản VND) chỉ vì hiểu logic code không validate — đây là bug thật tự tìm ra trong dự án, không phải giả định

A.5 Test Documentation (IEEE 829-style)

TÀI LIỆU	TRẢ LỜI CÂU HỎI GÌ	TƯƠNG ỨNG TRONG DỰ ÁN
Test Plan	Kiểm thử cái gì, ai làm, khi nào, tiêu chí pass/fail	Mục 4 "Entry/Exit Criteria" trong <code>TEST_DESIGN.md</code>
Test Design Specification	Kiểm thử như thế nào , kỹ thuật nào	Toàn bộ <code>TEST_DESIGN.md</code>
Test Case Specification	Từng bước cụ thể, input, expected output	Bảng 146 test case, mục 7
Test Summary Report	Kết quả sau khi chạy — pass/fail, bug tìm được	<i>(chưa có trong bộ tài liệu — vì đây là thiết kế, chưa chạy full regression)</i>

A.6 Severity vs Priority — câu hỏi kinh điển hay bị hỏi

- **Severity** = mức độ NGHIÊM TRỌNG về mặt kỹ thuật/nghịệp vụ (bug ảnh hưởng bao nhiêu tới hệ thống).
- **Priority** = mức độ CẦN SỬA GẤP đến đâu (theo góc nhìn business/lich release).
- **Chúng độc lập với nhau!** Ví dụ kinh điển nên thuộc: lỗi chính tả trên logo trang chủ ngân hàng → Severity thấp (không ảnh hưởng chức năng) nhưng Priority cao (ảnh hưởng thương hiệu, phải sửa ngay).
- **Ví dụ thật lấy từ dự án:** bug "giao dịch khai `currency` khác tài khoản mà không quy đổi/báo lỗi" → **Severity cao** (dữ liệu tài chính sai, số dư sai) và **Priority cao** (ảnh hưởng tiền thật) → phải sửa ngay, và đã sửa (thêm validate khớp `currency`).

A.7 Bug Life Cycle

New → Assigned → Open (Dev đang sửa) → Fixed → Retest → Closed (hoặc Reopened nếu retest vẫn fail; hoặc Rejected / Duplicate / Deferred tùy trường hợp).

A.8 Black box / White box / Gray box

- **Black box:** chỉ biết input/output, không cần đọc code — hầu hết test case trong `TEST_DESIGN.md` (test qua API).
- **White box:** đọc code để thiết kế test — đây chính là cách **các mã lỗi trong bảng "Error Code Reference"** của `TEST_DESIGN.md` được xác định chính xác (trích thẳng từ code, không đoán).
- **Gray box:** kết hợp cả 2 — biết sơ đồ DB/kiến trúc nhưng test qua API. Toàn bộ cột "Kiểm chứng qua DB (SQL)" trong `TEST_DESIGN.md` là ví dụ gray box điển hình: test qua API (black box) nhưng xác minh thêm bằng truy vấn DB trực tiếp (cần hiểu schema).

PHẦN B — Kiến thức nghiệp vụ ngân hàng lõi (Core Banking Domain)

Đây là phần **phân biệt tester ngân hàng với tester thường**. Ai cũng học được lý thuyết test ở Phần A, nhưng hiểu đúng nghiệp vụ ngân hàng mới là thứ nhà tuyển dụng bank thật sự tìm.

B.1 Core Banking System là gì?

Hệ thống lõi xử lý toàn bộ giao dịch tài chính hàng ngày của ngân hàng: mở tài khoản, gửi/rút tiền, chuyển khoản, cho vay, tính lãi, sổ cái kế toán. **T24 (Temenos)** là 1 trong các core banking system phổ biến nhất thế giới — dự án này được xây theo tinh thần kiến trúc/nghiệp vụ tương tự (không phải bản sao chính thức) để luyện tập.

Câu trả lời mẫu khi được hỏi "Bạn biết gì về T24": *"Em chưa làm việc trực tiếp trên T24 thật, nhưng em đã tự xây một hệ thống mô phỏng đúng các module cốt lõi của nó: CASA (tài khoản thanh toán), tiền gửi có kỳ hạn, cho vay, sổ cái kế toán kép, và đặc biệt là cơ chế Maker-Checker — nên em nắm được bản chất nghiệp vụ, phần còn lại là làm quen giao diện/thao tác cụ thể của T24 thật."*

B.2 Sổ cái kế toán kép (General Ledger / Double-Entry Bookkeeping)

Nguyên lý: mọi giao dịch tiền phải ghi **2 bút toán** (1 Nợ/Debit, 1 Có/Credit) với **số tiền bằng nhau** — tổng Nợ toàn hệ thống **luôn luôn** bằng tổng Có. Đây là bất biến (invariant) quan trọng nhất của bất kỳ core banking system nào.

Ví dụ thật đã verify trong dự án — khách nộp tiền mặt vào tài khoản:

Nợ 1000 (Tiền mặt và ngân quỹ)	100.000đ
Có 2000 (Tiền gửi thanh toán KH)	100.000đ

Trả góp vay (tách gốc và lãi — đây là điểm hay bị hỏi xoáy vì cần hiểu bản chất kế toán):

Nợ 2000 (Tiền gửi thanh toán)	1.725.483,67đ	
Có 1100 (Dư nợ cho vay)	1.625.483,67đ	← phần GỐC, giảm tài sản "khoản phải thu"
Có 4000 (Thu nhập lãi cho vay)	100.000,00đ	← phần LÃI, ghi nhận DOANH THU

Câu hỏi hay bị hỏi: "Vì sao không thể gộp chung gốc và lãi thành 1 bút toán?" → Vì bản chất kế toán khác nhau: trả gốc là **giảm tài sản** (khoản khách nợ ngân hàng giảm), trả lãi là **ghi nhận doanh thu** (income) — gộp chung sẽ làm sai báo cáo P&L (lãi/lỗ) dù báo cáo bảng cân đối (balance sheet) tổng vẫn "trông" đúng.

Kỹ thuật test bất biến này: chạy `SELECT SUM(debit), SUM(credit) FROM gl_entries GROUP BY entry_side` sau **mọi** lô test — nếu lệch dù 1 xu là bug nghiêm trọng nhất có thể có trong core banking (tiền "biến mất" hoặc "sinh ra từ hư không").

B.3 CASA — Current Account / Savings Account (Tài khoản thanh toán/tiết kiệm không kỳ hạn)

Tài khoản có thể rút/nộp tự do bất cứ lúc nào, lãi suất thấp (hoặc 0%). Trong dự án là bảng `accounts` — có máy trạng thái `active/dormant/frozen/closed`.

Câu hỏi hay gặp: "Dormant và Frozen khác nhau thế nào?" → *Dormant* (không hoạt động) thường do khách không giao dịch lâu ngày, ngân hàng tự động chuyển — khách có thể tự kích hoạt lại. *Frozen* (đóng băng) thường do quyết định nghiệp vụ/pháp lý (ngghi ngờ gian lận, yêu cầu pháp luật) — cần thao tác chủ động để mở lại, không tự động.

B.4 Tiền gửi có kỳ hạn (Term Deposit) — module tính lãi khó nhất

- **Lãi đơn (Simple Interest):** $I = P \times r\% \times \text{số ngày} / \text{cơ sở năm}$. Không có chuyện "lãi ra lãi".
- **Lãi kép (Compound Interest):** lãi kỳ trước được nhập vào gốc để tính lãi kỳ sau — $I = P \times ((1+r\%/12)^n - 1)$ nếu ghép hàng tháng.
- **Day-count convention (cơ sở tính ngày):** `actual/365` hay `actual/360` — 2 ngân hàng dùng 2 chuẩn khác nhau cho cùng 1 giao dịch sẽ ra lãi khác nhau dù cùng lãi suất công bố! Đây là điểm rất hay bị hỏi để test khả năng đọc hiểu hợp đồng/quy định.
- **Rút trước hạn (Early Withdrawal) — case khó kinh điển:** dù khách "chỉ còn 1 ngày nữa là đáo hạn", theo thông lệ ngân hàng VN khách **KHÔNG** được hưởng lãi suất đã cam kết — chỉ được hưởng lãi suất **không kỳ hạn** (thấp hơn nhiều, tính lãi đơn). Đây là 1 trong những nghiệp vụ **tester bank hay bị hỏi xoáy nhất** vì rất dễ hiểu nhầm là "cứ rút sớm thì được ít lãi hơn theo tỉ lệ" — SAI, bản chất là đổi hẳn sang mức lãi suất khác.
- **Đáo hạn tự động tái tục (Auto-renewal):** khi đến hạn, nếu khách chọn tái tục, lãi được **nhập vào gốc** để mở sổ mới (gốc mới = gốc cũ + lãi) — tiền không thực sự "chảy" qua tài khoản thanh toán.

B.5 Cho vay (Lending) & Amortization

- **Amortization (khấu hao trả góp)** — phương pháp phổ biến nhất: **dư nợ giảm dần, trả đều mỗi kỳ** (French/Reducing Balance) — số tiền trả mỗi kỳ **bằng nhau**, nhưng tỉ lệ gốc/lãi trong mỗi kỳ **thay đổi**: kỳ đầu trả lãi nhiều/gốc ít (vì dư nợ còn cao), kỳ cuối ngược lại.
- **NPL (Non-Performing Loan / Nợ xấu):** khoản vay có kỳ trả góp quá hạn vượt ngưỡng quy định — chuẩn phổ biến (tương tự Basel) là **quá hạn > 90 ngày** thì phân loại nợ xấu. Đây là kiến thức bắt buộc phải biết khi test module Loan ở bất kỳ bank nào.

B.6 Maker-Checker (4-Eyes Principle) — CHỦ ĐỀ QUAN TRỌNG NHẤT khi phỏng vấn tester ngân hàng

Định nghĩa: một nghiệp vụ nhạy cảm (di chuyển tiền, đổi trạng thái tài khoản...) phải qua **2 người khác nhau**: người **nhập lệnh (Maker)** và người **phê duyệt (Checker)**. Maker **không bao giờ** được là Checker của chính lệnh mình vừa nhập.

Vì sao quan trọng? Đây là kiểm soát nội bộ (internal control) cơ bản nhất chống gian lận nội bộ — 1 nhân viên đơn lẻ không thể tự ý chuyển tiền ra khỏi ngân hàng dù có quyền truy cập hệ thống.

3 lớp bảo vệ đã cài đặt và TEST THẬT trong dự án (kể chi tiết trong phỏng vấn sẽ rất ấn tượng):

1. **Chặn tự duyệt** — so `checker_id === maker_id` → từ chối, dù checker đăng nhập hợp lệ.
2. **Chặn theo vai trò (RBAC)** — 1 user role `maker` dù **không phải người nộp lệnh** vẫn không được duyệt, vì họ không có quyền `checker`. (Đây là lỗi hỏng em tự phát hiện: ban đầu hệ thống chỉ chặn "tự duyệt" mà quên chặn theo vai trò — 1 user role `maker` khác vẫn duyệt được miễn không trùng ID. Đã bổ sung *enforcement role*.)
3. **Danh tính không thể giả mạo** — `maker_id / checker_id` lấy từ **JWT token đã xác thực**, không nhận trực tiếp từ request body (nếu không, ai cũng tự khai mình là ai cũng được).

Câu hỏi hay gặp: "Nếu Checker duyệt xong nhưng nghiệp vụ thực thi bị lỗi (vd hết tiền) thì sao?" → Hệ thống phải **hoàn tác trạng thái hàng đợi về pending** (không được để kẹt ở `authorized` trong khi tiền

chưa thực sự di chuyển) để Checker biết mà xử lý lại — đây là 1 edge case rất hay bị bỏ sót khi thiết kế test, và đã được cài đặt + test riêng (**TC-AUTH-016**).

B.7 Idempotency (Tính bất biến khi gọi lại) — vì sao ngân hàng phải có

Trong hệ thống banking, client (app/web) có thể **gọi lại (retry)** 1 request do timeout mạng — nếu không kiểm soát, có thể **trừ tiền/giải ngân 2 lần** cho cùng 1 lệnh của khách. Giải pháp: client gửi kèm 1 **Idempotency-Key** duy nhất; server lưu lại kết quả theo key đó — gọi lại đúng key + đúng nội dung sẽ trả về **kết quả cũ đã lưu**, không thực thi lại.

Bug thật đã tìm và vá trong dự án: endpoint tạo khoản vay (giải ngân tiền thật) ban đầu **không** bắt buộc Idempotency-Key trong khi các endpoint di chuyển tiền khác (giao dịch, tiền gửi, trả góp, quy đổi ngoại tệ) đều có — đây là ví dụ tuyệt vời để trả lời câu "kể về 1 bug bạn tự tìm ra bằng cách rà soát chéo giữa các module tương tự nhau" (không cần ai chỉ, tự nhận ra sự thiếu nhất quán).

B.8 KYC / AML (khái niệm nên biết dù dự án chưa làm sâu)

- **KYC (Know Your Customer):** xác minh danh tính khách hàng trước khi cho phép giao dịch đầy đủ — trạng thái điển hình **pending → verified/rejected**.
- **AML (Anti-Money Laundering):** chống rửa tiền — hệ thống giám sát giao dịch bất thường (vd giao dịch giá trị lớn bất thường) để cảnh báo, tương tự module **fraud_alerts** trong dự án (giao dịch vượt ngưỡng tự động tạo cảnh báo với **risk_score**).

B.9 COB / EOD (Close of Business / End of Day) — Batch xử lý cuối ngày

Ngân hàng thật **không** xử lý mọi nghiệp vụ real-time bằng tay — có 1 batch chạy tự động mỗi đêm để: tính lãi dồn tích, tự động đáo hạn các sổ tiết kiệm tới hạn, phân loại nợ xấu, chốt sổ trong ngày. Đặc điểm quan trọng: **mỗi bản ghi xử lý độc lập (resilient)** — 1 lỗi ở 1 khách hàng không được làm dừng cả batch của hàng triệu khách khác.

B.10 FX / Tỷ giá hối đoái

Khi quy đổi giữa 2 loại tiền tệ, ngân hàng dùng **tỷ giá as-of** (tỷ giá có hiệu lực tại 1 thời điểm cụ thể, thường lấy bản ghi gần nhất trước hoặc bằng ngày giao dịch — không phải tỷ giá "mới nhất tuyệt đối"). Một hệ thống chỉ dùng **1 đồng tiền nền (base/functional currency)** để ghi sổ cái — mọi ngoại tệ khác phải quy đổi về đồng tiền nền khi hạch toán, nếu không tổng Nợ/Có sẽ "so sánh táo với cam".

PHẦN C — Kiểm thử API & SQL cho Tester Ngân hàng

C.1 REST API Testing — kiến thức tối thiểu

HTTP METHOD	Ý NGHĨA	VÍ DỤ TRONG DỰ ÁN
GET	Đọc dữ liệu, không thay đổi state	GET /accounts/:id
POST	Tạo mới / thực hiện hành động	POST /transactions

HTTP METHOD	Ý NGHĨA	VÍ DỤ TRONG DỰ ÁN
PATCH	Cập nhật 1 phần	<code>PATCH /accounts/:id/status</code>

HTTP STATUS	Ý NGHĨA	KHI NÀO GẶP TRONG DỰ ÁN
200	Thành công (GET/PATCH)	
201	Tạo mới thành công	Sau <code>POST /transactions</code> , <code>POST /term-deposits</code> ...
400	Lỗi nghiệp vụ (business rule)	<code>INSUFFICIENT_BALANCE</code> , <code>ACCOUNT_NOT_ACTIVE</code>
401	Chưa xác thực (thiếu/sai token)	<code>UNAUTHORIZED</code>
403	Đã xác thực nhưng không đủ quyền	<code>FORBIDDEN</code> (sai role)
404	Không tìm thấy tài nguyên	<code>ACCOUNT_NOT_FOUND</code>
409	Xung đột dữ liệu	<code>IDEMPOTENCY_KEY_CONFLICT</code> , <code>CUSTOMER_EMAIL_TAKEN</code>
422	Dữ liệu đầu vào không hợp lệ (validation)	Sai schema, sai kiểu dữ liệu
500	Lỗi hệ thống không xác định	Bug thật chưa được xử lý tường minh

Câu hỏi hay gặp: "401 và 403 khác nhau thế nào?" → 401 = "*Tôi không biết anh là ai*" (chưa đăng nhập/token sai); 403 = "*Tôi biết anh là ai rồi, nhưng anh không được phép làm việc này*" (đã đăng nhập nhưng sai role). Ví dụ thật: gọi `/auth-queue/:id/authorize` không có token → 401; có token hợp lệ nhưng role `maker` → 403.

C.2 SQL cần biết cho Tester ngân hàng (không cần giỏi như Dev, nhưng phải đọc/viết được)

```
-- Đối chiếu số dư
SELECT balance FROM accounts WHERE account_id = 3;

-- Kiểm tra bất biến kế toán kép (câu lệnh QUAN TRỌNG NHẤT cần thuộc)
SELECT entry_side, SUM(amount) FROM gl_entries GROUP BY entry_side;

-- Join để tra bút toán theo transaction
SELECT a.code, e.entry_side, e.amount
FROM gl_entries e JOIN gl_accounts a ON a.gl_account_id = e.gl_account_id
WHERE e.transaction_id = 36;

-- Đếm để phát hiện double-post (kiểm thử idempotency)
SELECT COUNT(*) FROM transactions WHERE account_id = 3;
```

Vì sao Tester cần viết được SQL thay vì chỉ tin API response? Vì response API là do **chính code cần test** trả về — nếu code có bug ở tầng ghi dữ liệu nhưng "che" được ở tầng trả response, chỉ tin response sẽ không bao giờ phát hiện ra. Đây chính là kỹ thuật **Gray-box testing** (mục A.8).

C.3 Kiểm thử bảo mật cơ bản cho tester (không cần chuyên sâu pentest)

- Thử gọi API **không có token** → phải bị chặn (401).
- Thử gọi API với token của **role thấp hơn** để làm việc của role cao hơn → phải bị chặn (403).
- Thử **SQL Injection** cơ bản trong input text (vd `full_name: "'; DROP TABLE customers; --"`) → hệ thống dùng parameterized query (`:name` → `$1`) nên an toàn — nhưng vẫn nên test để xác nhận.
- Kiểm tra **response không rò rỉ dữ liệu nhạy cảm** — vd `GET /users` không được trả `password_hash`.

PHẦN D — Trình bày dự án này trong phỏng vấn

D.1 "Elevator pitch" 30 giây

"Em tự xây 1 hệ thống mô phỏng lõi ngân hàng theo kiến trúc kiểu T24 — không chỉ CRUD tài khoản/giao dịch cơ bản, mà có đủ các module khó thật sự của ngân hàng: sổ cái kế toán kép tự động hạch toán mọi giao dịch, tiền gửi có kỳ hạn với lãi đơn/lãi kép/phạt rút trước hạn, cho vay có lịch trả góp, cơ chế Maker-Checker kiểm soát kép có đăng nhập JWT thật, và batch xử lý cuối ngày tự động. Em vừa là người phát triển vừa tự viết bộ 146 test case + kiểm chứng bằng SQL trực tiếp, nên em hiểu rõ vì sao mỗi rule nghiệp vụ lại được thiết kế như vậy, không chỉ học thuộc."

D.2 Câu hỏi follow-up chắc chắn sẽ gặp + cách trả lời

Q: "Bạn test bằng tool gì?" A: API test qua Node script/curl (mô phỏng CI), tài liệu API bằng Swagger UI tự sinh (OpenAPI), thiết kế test case theo IEEE 829, kiểm chứng chéo bằng SQL trực tiếp trên Postgres. Chưa dùng framework automation (Postman/Newman, Playwright cho UI) cho bộ test này — nói thật, đừng nhận đã làm automation nếu chưa.

Q: "Bug khó nhất bạn từng gặp trong dự án là gì?" (xem Phần E — có sẵn 3 câu chuyện thật)

Q: "Nếu sắp giao bạn test tính năng chuyển tiền quốc tế mới, bạn sẽ bắt đầu từ đâu?" A: (1) Đọc kỹ SRS/business rule trước — đặc biệt quy tắc tỷ giá as-of, đồng tiền nền; (2) Liệt kê equivalence class: cùng currency vs khác currency, có tỷ giá vs không có; (3) Boundary: tỷ giá đúng ngày vs không có bản ghi; (4) Thiết kế state transition nếu có trạng thái giao dịch; (5) Test bất biến — sau giao dịch, tổng tiền 2 hệ thống (gửi/nhận) phải khớp sổ cái; (6) Test bảo mật — ai được phép khởi tạo, có cần Maker-Checker không.

Q: "Bạn hiểu gì về double-entry accounting? Vì sao ngân hàng bắt buộc dùng?" A: Trả lời theo mục B.2 — nhấn mạnh đây là **cơ chế tự kiểm tra (self-checking)**: nếu code có bug làm sai số tiền, xác suất rất cao là bút toán sẽ **lệch cân** (`total_debit ≠ total_credit`) — tester chỉ cần 1 câu SQL để phát hiện hàng loạt lỗi bug tài chính mà không cần biết trước bug nằm ở đâu.

Q: "Sự khác biệt giữa test 1 hệ thống thông thường và hệ thống ngân hàng?" A: Hệ thống ngân hàng có thêm các lớp mà app thường không có: **kiểm soát nội bộ** (Maker-Checker), **bất biến kế toán** (trial balance), **audit trail đầy đủ** (mọi thay đổi tiền phải truy được ngược ai làm/khi nào), **idempotency** (không được trừ tiền 2 lần dù mạng lỗi), và **tuân thủ quy định** (KYC/AML/phân loại nợ xấu theo chuẩn quốc tế) — tester phải test cả nghiệp vụ **lấn** các lớp kiểm soát này, không chỉ chức năng bề mặt.

PHẦN E — 3 câu chuyện bug thật để kể (định dạng STAR: Situation-Task-Action-Result)

E.1 Bug ngày tháng do thư viện driver DB (kỹ thuật, chứng minh khả năng debug sâu)

- **Situation:** Sau khi thêm module tính lãi theo ngày, kết quả tính sai lệch — ngày bắt đầu hiển thị lùi lại 1 ngày so với thực tế đã lưu.
- **Task:** Tìm nguyên nhân gốc, không chỉ vá triệu chứng.
- **Action:** Nhận ra thư viện driver Postgres (`pg`) mặc định tự chuyển cột kiểu `DATE` thành object `Date` của JavaScript, và khi serialize sang JSON lại quy đổi theo giờ UTC — nếu server chạy ở múi giờ +7, "00:00 ngày X" bị hiểu thành "17:00 ngày X-1" giờ UTC, gây lệch ngày khi hiển thị (dù giá trị thật trong DB không sai).
- **Result:** Sửa tận gốc bằng cách đăng ký lại type parser cho driver để giữ nguyên chuỗi ngày thô thay vì qua object Date — sửa 1 chỗ nhưng khắc phục đúng cho **toàn bộ hệ thống** (không chỉ module đang làm), vì đây là bug ở tầng hạ tầng dùng chung.

E.2 Lỗi hổng nghiệp vụ tự phát hiện (chứng minh tư duy domain, không chỉ code)

- **Situation:** Sau khi hoàn thành cơ chế Maker-Checker (chặn tự duyệt), tưởng đã đủ an toàn.
- **Task:** Tự đặt câu hỏi "còn cách nào khác để bypass không?" thay vì dừng lại khi test case đầu tiên pass.
- **Action:** Nhận ra hệ thống có field `role` (`maker/checker/both/admin`) nhưng **chưa từng kiểm tra** — 1 user role `maker` (không phải người nộp lệnh gốc) vẫn tự do duyệt lệnh của người khác, miễn không trùng ID. Bổ sung kiểm tra role bắt buộc ở cả 2 phía (enqueue/authorize).
- **Result:** Phát hiện và vá 1 lỗ hổng kiểm soát nội bộ **trước khi** có ai khác chỉ ra — đúng tinh thần "critical thinking" mà tuyển dụng QA ngân hàng luôn tìm kiếm.

E.3 Sự thiếu nhất quán giữa các module tương tự (chứng minh khả năng rà soát hệ thống)

- **Situation:** Hệ thống có 5 endpoint di chuyển tiền thật (giao dịch, mở/rút/đáo hạn tiền gửi, trả góp vay, quy đổi ngoại tệ, tạo khoản vay).
- **Task:** Khi viết tài liệu đặc tả, chủ động đối chiếu chéo xem các endpoint "họ hàng" có được xử lý nhất quán không.
- **Action:** Phát hiện `POST /customers/:id/loans` (giải ngân) là endpoint **duy nhất** trong nhóm không bắt buộc `Idempotency-Key`, dù cũng di chuyển tiền thật như 4 endpoint còn lại.
- **Result:** Bổ sung, viết test case xác nhận (gọi lại đúng key → không giải ngân 2 lần), và cập nhật tài liệu — quy trình: *phát hiện* → *sửa* → *viết test* → *verify test pass* → *cập nhật tài liệu*, đúng vòng đời xử lý bug chuẩn.

PHẦN F — Câu hỏi tình huống (Scenario-based) thường gặp khi phỏng vấn Tester Bank

F.1 "Khách hàng báo số dư tài khoản sai sau khi chuyển khoản, bạn điều tra thế nào?" → (1) Xác định `transaction_id` liên quan; (2) `SELECT * FROM transactions WHERE account_id=... ORDER BY txn_timestamp` để xem lịch sử; (3) Đối chiếu `SELECT SUM(...)` tính lại balance từ đầu so với `accounts.balance` hiện tại (dự án này thậm chí có sẵn API `balance-history` suy diễn từ transactions cho đúng mục đích này); (4) Kiểm tra `gl_entries` tương ứng có cân bằng không; (5) Kiểm tra có bị double-post do lỗi idempotency không.

F.2 "Làm sao bạn biết 1 test case là đủ, không cần viết thêm?" → Không có "đủ" tuyệt đối — dùng kỹ thuật risk-based: ưu tiên P1 cho nghiệp vụ di chuyển tiền + bảo mật, P2 cho validation, P3 cho case phụ. Đối chiếu coverage với **toàn bộ nhánh business rule** trong tài liệu đặc tả (SRS) — nếu 1 rule trong SRS chưa có test case tương ứng, đó là gap.

F.3 "Bạn ưu tiên test cái gì trước khi release nếu chỉ có 1 ngày?" → P1: bất biến kế toán (trial balance), idempotency (chống double-post), bảo mật Maker-Checker (chặn tự duyệt/sai role), rồi mới đến happy-path của từng module.

F.4 "Automation hay Manual testing quan trọng hơn cho hệ thống ngân hàng?" → Không cái nào "quan trọng hơn" — Automation tốt cho regression lặp lại nhiều lần (vd chạy lại 146 test case mỗi lần release), Manual/Exploratory tốt cho phát hiện case mới, đặc biệt case nghiệp vụ tinh vi cần tư duy domain (như rút trước hạn sổ tiết kiệm) mà automation script chỉ chạy đúng-những-gì-được-viết-sẵn, không tự "nghĩ ra" case mới.

PHẦN G — Thuật ngữ cần thuộc nhanh (Glossary)

THUẬT NGỮ	GIẢI THÍCH NGẮN
CASA	Current Account / Savings Account — tài khoản thanh toán/tiết kiệm không kỳ hạn
GL	General Ledger — sổ cái kế toán kép
COA	Chart of Accounts — hệ thống tài khoản kế toán
Debit/Credit (Nợ/Có)	2 bên của 1 bút toán kế toán — không phải nghĩa "ghi nợ/tín dụng" thông thường
NPL	Non-Performing Loan — nợ xấu
KYC	Know Your Customer
AML	Anti-Money Laundering
Maker-Checker / 4-eyes	Kiểm soát kép — 2 người khác nhau nhập và duyệt
COB / EOD	Close of Business / End of Day — batch xử lý cuối ngày
Idempotency	Gọi lại nhiều lần cho kết quả như gọi 1 lần
As-of	Giá trị có hiệu lực tính đến 1 thời điểm cụ thể
RBAC	Role-Based Access Control — phân quyền theo vai trò
Amortization	Khấu hao/phân bổ trả góp dần
Day-count convention	Quy ước tính số ngày/năm khi tính lãi (actual/365, actual/360...)

THUẬT NGỮ	GIẢI THÍCH NGẮN
Reconciliation	Đối soát — so khớp dữ liệu giữa 2 nguồn (vd giữa core banking và hệ thống thẻ)
Trial Balance	Bảng cân đối thử — kiểm tra tổng Nợ = tổng Có

PHẦN H — Việc nên làm tiếp để phỏng vấn tự tin hơn

1. Đọc lại `TEST_DESIGN.md` và tự hỏi: "Nếu phỏng vấn hỏi module X, mình kể được test case nào ấn tượng nhất?"
2. Tự thực hành nói to phần D.1 (elevator pitch) và Phần E (3 câu chuyện bug) — kể trôi chảy trong 1–2 phút mỗi câu chuyện, không đọc.
3. Ôn thêm nếu công ty target dùng cụ thể T24 thật: tìm hiểu thuật ngữ riêng của T24 (AA - Arrangement Architecture, COB thật của T24 khác gì batch demo ở đây) — vì mức độ phức tạp của T24 production lớn hơn rất nhiều so với bản demo này, nên khiêm tốn khi so sánh.
4. Chuẩn bị câu hỏi ngược lại cho nhà tuyển dụng (luôn được đánh giá cao): "Bên mình dùng core banking system nào, quy trình QA có tách riêng UAT với Regression không, đội QA có phải viết SQL kiểm chứng trực tiếp như em từng làm không?"